

STATE//LESS AI 활용 기술 문서

0. 요약 — AI를 코드 생성기가 아니라 판단 파트너로 사용한 세 장면

이 문서의 핵심은 "AI에게 무엇을 만들게 시켰는가"가 아니라, AI를 통해 무엇을 검증하고 어떻게 판단을 바꿨는 것입니다. 개발 기간 동안 같은 패턴이 세 번 반복됩니다: 결과물을 의심 → AI로 근거를 만들거나 검증 → 그 근거로 실제 설계를 바꿈. 대표 사례 세 가지:

- "재미없다"는 판단을 감이 아니라 이론으로 검증:** SIGNAL LOCK 메커닉이 실제 플레이에서 재미없다는 평가를 받자, 코드를 고치기 전에 먼저 MDA·Koster·Swink·Csikszentmihalyi·Salen & Zimmerman 5개 게임 디자인 이론을 조사해 현재 게임의 약점을 각 이론에 정확히 대응시켰습니다(\$2 "재미 이론 기반 전면 재설계"). 그 진단 결과로 핵심 메커닉을 SIGNAL TRIAGE 로 전면 교체했습니다.
- "재밌어졌다"는 주장을 AI 서버에이전트의 실제 플레이로 검증:** 재설계 후 재미 여부를 다시 감으로 판단하지 않고, 화면 녹화 없이 텍스트 상태(JSON)만으로 게임을 조작할 수 있는 하네스를 직접 만들어 서버에이전트가 실제로 6웨이브를 플레이하고 평가하게 했습니다. 이 과정에서 사람보다 느린 LLM 반응속도가 게임 판정을 왜곡하는 문제(가상 시계로 해결)까지 직접 발견하고 고쳤습니다(\$2 "서버에이전트 실패 테스트").
- AI가 찾아낸 이슈를 그대로 믿지 않고 소스 코드로 재검증:** 서버에이전트가 보고한 버그 주장들을 곧바로 반영하지 않고 main.js 의 실제 코드를 하나씩 대조해 사실 여부를 확인한 뒤에만 반영했습니다. 이 검증 과정에서 두 건은 실제 버그로 확인되어 수정했고(오답 대사 고정 오류, ARCHIVE LENS 설명 불일치), 나머지는 "의도된 설계"로 판단해 이번 범위에서 제외했습니다.

1. AI 활용 개요

개발 보조 도구로 OpenAI Codex를 사용했습니다. AI는 게임 콘셉트 구체화, 프론트엔드 구현, 게임 로직 테스트, 접근성 점검, 문서 초안 작성에 활용했습니다. 최종 요구 방향은 참여자가 직접 제시했으며, AI가 제안한 초기 아케이드 콘셉트는 참여자의 피드백에 따라 폐기하고 메타 퍼즐로 다시 설계했습니다.

런타임 게임 자체는 외부 생성형 AI API를 호출하지 않습니다. 심사 환경에서 API 키, 비용, 응답 지연, 네트워크 장애에 의존하지 않도록 브라우저 상태와 결정적 규칙으로 "로컬 AI 추론"을 구현했습니다.

2. 주요 프롬프트와 반영 내역

최초 요구

사전과제 폴더 안의 내용을 확인하고, 사전과제 안에 게임 작업을 완료해보자. 내가 원하는 요소는 키보드 혹은 마우스로만 플레이 가능하고 웹으로 서비스되는 것 하나뿐이다.

반영 내용:

- 정적 웹 빌드로 구현
- 키보드 전용 경로와 마우스 전용 경로를 각각 제공
- 시작, 본 플레이, 결말 선택, 재시작까지 어느 한 입력 장치만으로 완주 가능
- GitHub Pages 자동 배포 워크플로 포함

기술 방향 피드백

최근 AI에 나타난 프론트엔드 특이점, 특히 Pretext 등을 이용해서 메타적(쿠키 상태)인 게임은 어떨까. 그 외에도 최신 기술을 다룬다면 좋겠다.

반영 내용:

- 2026년 공개된 Pretext를 실제 게임 보드의 텍스트 배치 엔진으로 사용
- 1st-party 쿠키의 방문·완료 런·보존 정책·이전 결말·최고 점수·기억 조각을 서사 및 문제 생성에 사용
- Cookie Store API, View Transition API, Page Visibility API, BroadcastChannel, ResizeObserver, CSS Container Queries를 장식이 아닌 게임 상태와 상호작용에 연결
- 개인정보나 다른 출처의 쿠키를 읽지 않는 로컬 전용 구조 채택

게임성·캐릭터 피드백

캐릭터의 부족, 게임을 계속하는 동기의 부족, 질문의 다양성 부족. 지금 게임 내에서는 처음에 말했던 프론트엔드 혁신이 딱히 구현되어 있지 않은 것 같다.

계속해서 작업해보자. 이건 게임이야.

반영 내용:

- 브라우저 안에서 깨어난 성인 여성 기억 관리 AI **MORI** 를 주인공으로 설정
- 기본 캐릭터 이미지 생성을 위한 색상·성격·성별·해상도·파일 형식·용량·네거티브 프롬프트 작성
- 기존 단일 거짓 찾기를 단일 신호·참/거짓 판별·값 복원·2필드 교차검증·3필드 체크섬의 6종 규칙으로 확장
- 결과 저장 시 최대 6개의 기억 조각을 모으는 재방문 목표 추가
- 정답을 그대로 보여주던 상태 칩을 제거하고 간접적인 브라우저 로그로 교체
- Pretext가 증거 로그를 중앙 오염 코어 주위로 직접 흐르게 해, 화면 폭에 따라 플레이 화면이 실제로 재구성되도록 변경
- 범용 사이버펑크 인상을 줄이고 **MEMORY:// LOCAL ARCHIVE** 일지 UI로 재설계
- UI에서 먼저 전신 파일과 플레이 반응 슬롯을 정의한 뒤, 17개 상황 상태와 마스킹 편집 프롬프트를 설계
- 여섯 라운드 성공 여부를 누적 표시하는 **VERIFIED GATE**, 5초 위기 구간, 규칙·정답·실수에 반응하는 MORI 대사를 추가해 게임 세션의 긴장과 캐릭터 피드백을 강화
- 플레이 시작 시 소개 열을 접는 몰입 모드, 0.1초 타이머, MORI 판정 컷인, 클릭 또는 **Enter / Space** 로 직접 다음 라운드에 진입하는 조작을 추가해 “페이지 안의 데모” 인상을 줄임
- 기존 브라우저 신호 10개에 쿠키 진행도 **SHARDS·RUNS·RETENTION·BEST** 를 추가하고 다음 **MORI.LOG** 보상을 HUD에 노출해 재플레이 문제 풀과 목표를 강화
- 기본 규칙 4개 뒤 고난도 규칙 2개라는 곡선은 유지하면서 각 묶음의 순서를 시드별로 바꾸고, 6개 파일 완성 뒤에는 다음 파일 대신 최고 점수·6연속 **SYNC** 를 제시하도록 엔드게임 목표를 교체
- 실수 뒤 3연속 정답으로 이전 오류와 무결성 1칸을 한 번 복구하는 **SYNC RECOVERY** 를 추가해, **VERIFIED GATE** 가 닫힌 뒤에도 남은 라운드를 이어갈 컴백 목표를 제공
- 매 라운드 정답 350점과 실패 시 무결성 2칸을 거는 **DEEP VERIFY** 를 추가하고, 클릭 또는 **D** 로 같은 위험 선택을 조작하도록 구현
- 마지막 고난도 문제를 전용 경로·MORI 대사·코어 상태색·Web Audio 신호를 가진 **FINAL CORE** 로 전환하고, 이 라운드의 **DEEP VERIFY** 보너스만 700점으로 올려 한 판의 절정을 설계
- 완료 런·조각 수에 따라 **SYNC CHAIN·WAGER PROOF·CLEAN ARCHIVE** 가 순환하는 **MORI REQUEST** 를 추가하고, 진행도·+600점·**REQUEST SEALED** MORI 반응을 인트로부터 결과까지 연결

손맛 재설계 피드백

게임 테마가 무슨, 쿠키 로그 해석게임이 된것같은데,,, 이게 맞냐

게임 기획부터 다시하자, 특히 nhn ai 게임제작 해커톤이란점을 명심해, 처음부터 다시 만들어도됨. 지금은 게임같지도않아,

듀오링고같아.

또한, 사전지식이 없어도 즐길수있는게임으로 해

반영 내용:

- 제출 요강(사전과제.txt)을 다시 확인해 장르·메커닉 제약이 없음을 확인. NHN 게임 계열(한게임 등)이 강점을 가진 타이밍 기반 손맛 방향으로 방향을 좁힘
- 원인 분석: 6가지 라운드 "종류"가 있어도 실제 손동작은 항상 "읽고 아무 때나 클릭"으로 동일해 시간차·스킬이 없었음. 지식 퀴즈 앱과 구조적으로 동일했던 것이 "듀오링고 같다"는 인상의 원인
- 정답 확정을 즉시 채점하지 않고, 좌우로 오가는 마커를 정확한 타이밍에 다시 눌러 확정하는 **SIGNAL LOCK** 미터를 도입. PERFECT/GOOD/MISS로 정답 여부와 별개로 보상이 갈리게 해 "정답을 아는 것"과 "확정하는 것" 사이에 실력차를 만들
- "사전지식 없이도 즐길 수 있어야 한다"는 요구에 따라, 라운드 종류별로 다르던 지시문(**prompt / instruction**)을 라운드가 바뀌어도 항상 같은 한 문장("지금 진짜인 신호를 고르세요")으로 통일. 6종 라운드의 실제 판정 방식은 이미 전부 "세 후보 중 하나를 고른다"로 귀결되는 구조였기 때문에, 규칙 학습 부담만 제거하고 문제 생성 로직(**createRoundDeck**)은 전혀 바꾸지 않음
- 무결성·SYNC·ARCHIVE LENS·DEEP VERIFY·MORI REQUEST·기억 조각 시스템, MORI 캐릭터·대사·17개 표정 이미지는 이미 잘 만들어진 부분이라 손대지 않고 그대로 재사용

재미 이론 기반 전면 재설계 피드백

음,,, 좀더 ㄱㄱ 재미있게 하자. 또한,, 쿠키 기반 기억 수집은, 아직도 일반인들, 또한 게임 심사자들에게는 그냥, 코드 해석의단계에 머무를것같아, 좀더 재미있게 해보자, 또는 게임자체를 처음부터 만들어버려도돼. 또한 재미있는게임 개발론도 정리해보고 가보자.

SIGNAL LOCK 을 붙인 뒤에도 실제 플레이 결과 "여전히 재미없다"는 평가를 받았고, 이번에는 손보기 전에 실제 게임 디자인 이론을 먼저 조사해 진단 근거로 삼으라는 요구를 받음. 웹 검색으로 다음 다섯 이론을 정리하고 각각을 현재 게임의 구체적 약점에 대응시킴.

이론	핵심 주장	현재 게임에 대응되는 약점
MDA 프레임워크 (Hunicke·LeBlanc·Zubek)	Mechanics(규칙)가 Dynamics(플레이 중 실제 행동 패턴)를 낳고, 그 Dynamics가 Aesthetics(플레이어가 느끼는 재미)를 낳는다	6종 라운드라는 Mechanics 표면이 달라 보여도, 실제 Dynamics는 항상 "읽고 → 아무 때나 하나 고르고 → 확정"으로 단 하나뿐이었음
Raph Koster, 『재미 이론(A Theory of Fun for Game Design)』	재미는 패턴을 인식하고 숙달하는 과정에서 나오며, 패턴이 완전히 학습되면 재미가 마른다	14개 브라우저/쿠키 신호는 몇 판만 반복하면 금방 암기되어 패턴이 남지 않음
Steve Swink, 『Game Feel』 (저지/juice 개념)	실시간·연속적인 입력·반응 루프와 시청각 과잉반응(juice)이 조작 자체의 쾌감을 만든다	정답을 "아무 때나 클릭"하는 구조에는 실시간성이 전혀 없어 손맛(juice)이 들어갈 자리가 없었음
Mihaly Csikszentmihalyi, 몰입 (Flow) 이론	몰입은 난이도와 실력이 팽팽히 맞서고, 목표가 명확하고, 피드백이 즉각적일 때 발생한다	제한 시간이 있어도 클릭 타이밍에 실력 차이가 반영되지 않아 난이도·실력 곡선이 사실상 평평했음

Katie Salen & Eric Zimmerman, 『Rules of Play』의 Meaningful Play	좋은 선택은 결과가 눈에 보여야 (discernible) 하고, 그 결과가 이후 플레이와 실제로 얹혀야(integrated) 한다	6판 중 5판만 맞히면 다음 스토리가 그냥 풀리는 구조는 결과는 보이지만(discernible) 플레이 방식과는 무관(비integrated)했음 — "쿠키 기반 기억 수집이 코드 해석 단계에 머무른다"는 지적이 정확히 이 지점
--	--	--

이 진단에 따라 AskUserQuestion 으로 네 가지 핵심 메커닉 후보(실시간 신호 트리야지 / 순서 재배열 퍼즐 / 자원 배분 협상 / 패턴 암산)를 제시했고, 참여자가 **실시간 신호 트리야지**(Overcooked·Diner Dash류의 실시간 분류 압박)를 선택함.

반영 내용:

- 핵심 메커닉을 "세 후보 중 하나를 고르고 타이밍 미터로 확정"에서 "화면을 계속 가로질러 흐르는 증거 칩을 실시간으로 TRUE / FALSE / CORRUPTED 세 구역에 분류"하는 SIGNAL TRIAGE 로 전면 교체. 웨이브가 진행될수록 스폰 속도·동시 등장 수·판독 불가 칩 비율이 함께 늘어나 Flow 이론의 난이도·실력 곡선을 명시적으로 설계
- 각 신호(createFactCatalog)가 이미 갖고 있던 truthText / lieText / decoyText 세 필드를 그대로 세 구역의 칩 원본으로 재사용 — 문제 콘텐츠는 한 글자도 새로 쓰지 않고 메커닉만 교체
- 메타 진행(6라운드→6웨이브, 무결성/SYNC/ARCHIVE LENS/DEEP VERIFY/MORI REQUEST/기억 조각)은 전부 "웨이브별 정답/오답/놓침 개수와 누적 score/streak/integrity"라는 동일한 집계 값만 소비한다는 점을 확인하고 그대로 재사용 — 웨이브 내부 채점 로직만 새로 만들고 그 결과를 기존 필드에 그대로 채워 넣음
- "코드 해석 단계에 머무른다"는 지적에 대한 저비용 보완으로, 결과 집계 시 플레이 통계(회복 발동 여부· DEEP VERIFY 승리 횟수·무결점 여부)로 RECOVERY / RISK / PRECISION / SPEED 네 가지 런 스타일 태그를 계산하는 getRunStyleTag 를 추가하고, 결과 화면 MORI 대사에 스타일별 한 줄을 덧붙여 "어떻게 플레이했는지"가 결과 텍스트에 실제로 반영되도록 함 — 장비·빌드 시스템 같은 전면적 구조 변경 대신, 이미 있는 통계를 서사에 연결하는 최소 개입으로 판단
- 실시간 칩 흐름이 도입되며 정적 텍스트 리플로 엔진이던 Pretext(prepareWithSegments / layoutNextLine , rich-inline 선택지 배치)의 실사용처가 사라짐을 확인 — src/pretext-layout.js 삭제, main.js 의 관련 import 제거, package.json 에서 @chenglou/pretext 의존성 제거. 조용히 빼지 않고 이 문서와 UI_UX_아트디렉션.md 에 제거 사유를 기록
- 놓친 칩(시간 초과로 흘러 나감)은 연속 기록만 끊고 무결성은 깎지 않도록 설계해 "판단을 잘못된 것"과 "미처 손이 못 간 것"을 구분 — 실시간 압박 자체가 이미 난이도를 담당하므로 이중 페널티로 좌절감을 늘리지 않기 위함
- 마우스 조작은 최초 검토했던 연속 드래그-드롭 대신 클릭으로 칩을 선택하고 클릭으로 구역을 지정하는 방식으로 구현 — 키보드(화살표로 칩 선택 + 숫자 키로 구역 지정)와 동일한 선택→확정 모델을 공유해 두 입력 경로의 조작 난이도를 완전히 동등하게 유지하기 위한 의도적 단순화이며, 터치 입력에도 그대로 대응됨
- prefers-reduced-motion 에서는 칩이 화면을 가로지르는 애니메이션 대신 고정된 세로 카드 스택 + 너비가 줄어드는 막대로 동일한 타이밍 정보를 전달하도록 구현 — 스폰/만료 타이밍과 목표 개수는 동일하게 유지해 움직임에 민감한 플레이어도 난이도 손해 없이 완주 가능

서브에이전트 실플레이 검증과 LIVE SIGNAL 패널

지금상태 체크 ㄱ / 라이브 url에서 보이는 게임상태가 가장최신상태인거지? 이 게임이,, 재밌다고 생각해?

음,,, 좀더 ㄱㄱ 재미있게 하자 ... 게임 재미, 대중성, 가시성 등을 위주로 작업 부탁해.

SIGNAL TRIAGE 재설계가 실제로 재밌는지는 코드만 읽어서는 판단할 수 없어, **화면 녹화 없이** 서브에이전트가 실제로 플레이해 평가하도록 검증 하네스를 만들었다.

- **텍스트 전용 하네스**: puppeteer-core 로 헤드리스 Chrome을 띄우고, DOM 상태(칩 텍스트, 점수, 무결성, 웨이브 진행도, MORI 대사 등)를 JSON으로만 노출하는 로컬 HTTP 서버를 작성. 서브에이전트는 스크린샷/영상을 전혀 보지 않고 GET /state / POST /action 왕복만으로 플레이함 — 토큰 비용이 큰 시각적 검증 없이 실제 판정 로직까지 검증 가능.
- **가상 시계 문제 해결**: 초반 웨이브도 23초·칩 하나당 7.2초로 사람 반응속도 기준이라, LLM이 HTTP 왕복으로 판단하면 인간보다 훨씬 느려 대부분의 칩을 놓치고 "불가능하다"는 오탐이 나올 뻔했다. page.evaluateOnNewDocument() 로

`performance.now()` / `requestAnimationFrame` / `cancelAnimationFrame` 을 게임 스크립트가 로드되기 전에 가상 시계로 치환해, `POST /advance {ms}` 를 호출할 때 게임 시간이 흐르도록 만들 — 에이전트의 사고 시간이 아무리 길어도 게임 내 시간은 명시적으로 진행시킨 만큼만 흐름. 실제 게임 코드는 한 줄도 건드리지 않고 하네스 쪽 프리로드 스크립트만으로 해결.

- **검증 결과:** 서버에이전트가 마우스 런(6웨이브 풀클리어, 5랭크) + 키보드 런(풀클리어 1회, 의도적 실패 2회)을 실제로 진행하며 "이전 정적 4지선다 방식보다 확실히 더 재밌다"고 평가. 동시에 소스 대조로 확인된 구체 이슈 두 가지: (1) 오답 시 MORI 대사가 실제 정답 구역과 무관하게 항상 "그건 오염된 쪽이야"로 고정 출력(`main.js`), (2) `ARCHIVE LENS` 사용 대사가 "50%로 좁혀준다"는 인상을 주지만 실제로는 정답 구역을 100% 노출.
- **근본 원인 진단:** 위 플레이테스트와 별개로, Salen & Zimmerman의 "discernible"(선택의 결과가 눈에 보여야 한다) 기준으로 다시 보면 더 큰 문제가 있었다 — 칩이 참인지 판단할 근거(테마/시간/입력기기/뷰포트 등 환경 신호) 자체가 화면 어디에도 노출되지 않아, 소스를 모르는 첫 플레이어는 웨이브 3~4까지도 사실상 추측으로 플레이하게 되는 구조였다.
- **반영:** `createFactCatalog` 가 이미 계산해 두던 14개 사실의 `label / value` (예: `TIME` → `NIGHT`, `VIEWPORT` → `WIDE`)를 그대로 나열하는 `LIVE SIGNAL` 패널을 플레이 화면에 추가(`renderLiveSignalPanel`, `createDeck()` 에서 호출) — 새 로직 없이 이미 있는 값만 노출. 오답 MORI 대사는 `pulseMoriState(state, dialogueOverride)` 로 확장해 실제 정답 구역(`chip.def.zone`)에 맞는 3종 문구로 분기, `ARCHIVE LENS` 대사도 실제 동작(정답 구역 100% 노출)에 맞게 수정. 첫 런의 첫 웨이브에만 "칩 문장을 `LIVE SIGNAL`과 비교하라"는 1회성 토스트로 온보딩을 보강. 그 외 발견된 이슈(키보드 모드 결과화면 자동 스킵 위험, 스트릭 표시-보너스 캡 불일치, `ARCHIVE LENS` 희소성)는 참여자가 "버그보다 재미/가시성 위주"라고 명시해 이번 패스에서는 보류.

3. AI가 지원한 구현 영역

기획

- HTTP의 무상태성과 쿠키의 상태성을 핵심 모순으로 설정
- 실제 브라우저 텔레메트리 14개 사실을 실시간 신호 트리아지로 흘러보내는 6웨이브 게임 루프 설계
- 재방문, 탭 이탈, 복수 탭 감지를 메타 서사로 연결
- 기억 조각 6개를 쿠키에 저장하는 중기 진행 목표 설계

개발

- Vite 기반 정적 웹 프로젝트 구성
- 키보드 화살표 선택 + 숫자 구역 지정과 마우스 클릭 선택 + 클릭 구역 지정을 동일한 선택→확정 모델로 통합 구현
- 웨이브별 목표 칩 수·제한 시간, 연속 정답 보너스, 무결성, 랭크 판정 구현
- 조각 수에 따라 성장하는 제한 자원 `ARCHIVE LENS` (웨이브당 1회, 선택 중인 칩의 정답 구역 확인), 키보드 `F` 입력 구현
- `VERIFIED` 성공에만 기억 조각을 지급하고 조각마다 MORI 서사 파일을 공개하는 보상 루프 구현
- 연속 정답 `SYNC` 를 HUD와 MORI 반응 상태에 연결
- 3연속 정답 시 가장 최근 오류를 `🔄` 상태로 바꾸고 무결성을 회복하는 1회성 `SYNC RECOVERY`, 복구 전용 MORI 컷인과 A랭크 상한 구현
- 웨이브당 1회, 5초 동안 정답 추가 점수와 오답 2칸 손실이 적용되는 `DEEP VERIFY` 버스트, 저무결성 잠금, MORI 상태·대사·판정 컷인 구현
- 마지막 웨이브에서 `DEEP VERIFY` 보너스를 +700점으로 올리고 `CORE EXPOSED` MORI 상태와 코어 색상 연출을 연결하는 최종 웨이브 연출 구현
- 런별 도전의 순환·진행·단일 보너스 지급, 플레이 상태줄, 완료 컷인과 캐릭터 대사, 결과 판정을 연결하는 `MORI REQUEST` 구현
- `VERIFIED GATE` 에 현재 웨이브·성공·실패·보상까지 남은 목표 칩 수를 표시하고 더 이상 성공할 수 없는 런을 즉시 구분
- 웨이브 종료 시점 판정에서 경고색, Web Audio 신호, MORI 반응 상태를 동시에 전환
- Cookie Store 비동기 저장과 `document.cookie` 폴백 구현
- 칩 하나의 이동 진행률을 단일 CSS 커스텀 프로퍼티(`--chip-progress`)로 표현해, 일반 모드는 좌우 이동으로 `prefers-reduced-motion` 은 고정 스택 + 너비 축소 막대로 같은 값을 다르게 렌더링하는 구조 구현
- 외부 파일 없는 Web Audio 효과음 구현

- GitHub Pages Actions 워크플로 작성
- 칩 스폰·이동·만료를 담당하는 `requestAnimationFrame` 루프(`updateWaveFrame`)와, 정답/오답/타임아웃으로 웨이브를 종료하는 순수 판정 함수(`resolveChip`, `expireChip`, `concludeWave`)로 실시간 트리아지 메커니즘 구현. 씬 전환마다 다르던 상단 지시문을 웨이브가 바뀌어도 동일한 공통 상수(`PLAY_INSTRUCTION`)로 유지
- 플레이 통계(회복 발동 여부·`DEEP VERIFY` 승리 횟수·무결점 여부)로 4가지 런 스타일 태그를 계산하는 순수 함수 `getRunStyleTag` / `getRunStyleRemark` 구현, 결과 화면 MORI 대사에 반영
- 이미 계산되어 있던 `createFactCatalog`의 `label` / `value`를 그대로 나열하는 `LIVE SIGNAL` 패널(`renderLiveSignalPanel`) 구현 — 새 데이터/로직 없이 `createDeck()` 한 곳에서만 호출해 환경 변화 시에도 자동 갱신
- `pulseMoriState`에 `dialogueOverride` 매개변수를 추가해, 오답 시 MORI 대사가 실제 정답 구역(`chip.def.zone`)에 맞는 3종 문구 중 정확한 것으로 분기되도록 구현

검증

- 6개 웨이브 인덱스 전부에서 목표 칩 수·스폰 간격·이동 시간·동시 등장 수·판독 불가 비율이 배열 길이와 단조 방향을 지키는지(`getWaveConfig`) 테스트
- 동일 시드에서 `createChipPool`이 동일한 칩 구성을 재현하는지, 모든 칩의 구역이 `true` / `false` / `corrupted` 중 하나이고 텍스트가 해당 신호의 올바른 필드(`truthText` / `lieText` / `decoyText`)에서 왔는지 테스트
- 대량 샘플링으로 판독 불가(`corrupted`) 칩 비율이 웨이브가 진행될수록 실제로 늘어나는지 테스트
- 점수·난이도·결말 경계값 테스트
- `SYNC RECOVERY`가 3연속 이전에는 발동하지 않고 가장 최근 오류를 런당 한 번만 복구하며, 복구 런이 S랭크를 받지 않는지 테스트
- `DEEP VERIFY`의 기본 보너스 350점·마지막 웨이브 보너스 700점과 실패 시 무결성 2칸 손실 경계값 테스트
- `MORI REQUEST` 3종 순환과 완료 경계, +600점 단일 지급, `REQUEST SEALED` 상태와 결과 판정 테스트
- `getRunStyleTag`가 `RECOVERY` / `RISK` / `PRECISION` / `SPEED` 네 조건과 빈 통계 기본값을 각각 정확히 반환하는지 테스트
- 기존 버전 쿠키가 기억 조각 0개인 버전 2로 안전하게 이전되는지 테스트
- 쿠키 데이터 직렬화 시 허용 필드만 보존되는지 테스트
- 헤드리스 브라우저로 마우스 선택→구역 클릭, 키보드 화살표 선택→숫자 구역 지정 두 경로 모두 칩 분류가 정상 판정되는지, 웨이브 중 오답을 내도 웨이브가 즉시 끝나지 않고 목표 개수 도달 시에만 종료되는지 점검
- `prefers-reduced-motion`에서 고정 카드 스택 렌더링이 정상 동작하는지, `ARCHIVE LENS`가 선택된 칩의 정답 구역을 `data-reveal-zone`으로 정확히 표시하는지, `DEEP VERIFY` 버스트 활성화 구간이 `dataset.active`로 전환되는지, 6웨이브 전체를 진행해 결과 화면과 런 스타일 대사까지 콘솔 에러 0건으로 도달하는지 점검
- 텍스트 전용 가상 시계 하네스로 `LIVE SIGNAL` 패널의 각 행 값이 `GET /telemetry`가 읽은 실제 브라우저 상태와 일치하는지, 의도적 오답 시 MORI 대사가 실제 정답 구역에 맞는 문구로 나오는지, `ARCHIVE LENS` 사용 후 대사가 수정된 문구로 바뀌는지, 뷰포트 리사이즈로 패널이 실시간 갱신되는지 직접 점검

4. 핵심 기술 구조

SIGNAL TRIAGE 실시간 보드

`createChipPool(catalog, waveIndex, seed)`가 세션 시드로 14개 신호를 섞어 웨이브당 칩 풀을 만듭니다. 각 신호의 `truthText` / `lieText` / `decoyText` 세 필드가 그대로 `TRUE` / `FALSE` / `CORRUPTED` 세 구역 칩의 원본 텍스트가 되며, 웨이브별 판독 불가(`corrupted`) 비율(`WAVE_CORRUPTED_CHANCE`)만큼 오염 칩 비중이 늘어납니다.

칩의 화면상 위치는 스폰 시각부터 경과한 비율 하나(`--chip-progress`, 0~100)만으로 표현됩니다.

`requestAnimationFrame` 루프(`updateWaveFrame`)가 이 값을 매 프레임 갱신하고, CSS가 이 값을 일반 모드에서는 좌우 이동(`translateX`)으로, `prefers-reduced-motion`에서는 고정된 세로 카드 스택 옆의 너비 축소 막대로 각각 다르게 해석합니다 — 같은 타이밍 데이터를 두 가지 시각 언어로만 나눠 그리므로 별도의 JS 분기 없이 접근성 대응이 끝납니다.

선택→확정 입력은 마우스와 키보드가 같은 모델을 공유합니다. 클릭(또는 화살표 키)으로 활성 칩 하나를

`runtime.selectedChipId` 로 선택하고, 구역 버튼 클릭(또는 숫자 키 1/2/3)으로 확정합니다. 확정 시점의 판정은 순수 함수 `resolveChip / expireChip` 이 담당하며, 웨이브 종료 조건(목표 칩 수 도달·무결성 소진·제한 시간 만료)은 `concludeWave` 에서 한 곳으로 모아 처리합니다.

이전 버전에서 사용하던 `@chenglou/pretext` (정적 텍스트 리플로 엔진)는 이번 재설계로 플레이 화면이 정적 배치에서 실시간 칩 애니메이션으로 바뀌며 실사용처가 사라져 의존성 자체를 제거했습니다.

쿠키 상태 모델

단일 쿠키 `state_less_memory_v1` 에 다음 화이트리스트 필드만 저장합니다.

```
{
  "version": 2,
  "visits": 1,
  "runs": 0,
  "bestScore": 0,
  "lastEnding": null,
  "policy": "session",
  "fragments": 0
}
```

값은 읽은 뒤 형식·범위·허용 열거형을 다시 검증합니다. 최신 보안 컨텍스트에서는 이벤트 루프를 막지 않는 Cookie Store API를 우선 사용하고, 미지원 브라우저에서는 `SameSite=Strict` 인 `document.cookie` 로 전환합니다.

결정적 로컬 추론

방문·테마·시간·입력·탭·화면 폭·네트워크·이전 결말에 완료 런·보존 정책·최고 점수·기억 조각을 더한 14개 사실마다 참 문장 (`truthText`), 거짓 문장 (`lieText`), 제3의 오염 문장 (`decoyText`)을 정의합니다. 웨이브가 시작될 때 세션 시드로 이 14개 사실과 세 문장 필드를 섞어 웨이브별 칩 풀(`createChipPool`)을 만들고, 웨이브가 진행될수록 목표 칩 수·동시 등장 수·판독 불가 비율이 함께 늘어납니다. 같은 상태와 시드는 같은 칩 구성을 만들므로 테스트와 재현이 가능합니다.

캐릭터 이미지 프롬프트

`캐릭터/MORI_기본이미지_생성_프롬프트.md`에는 초기 성인 여성 설정, 양면적 성격, 색상, 전신 구도와 파일 규격을 기록했습니다. 참여자가 선택한 1024×1536 RGBA PNG는 실투명 알파로 확인 후 전신 기준으로 확정했습니다.

추가 피드백 이후 캐릭터보다 UI 설정을 먼저 확정했습니다. `docs/UI_UX_아트디렉션.md`에서 브라우저 로컬 기억 보관소의 경로·색인·보존·정정 언어를 정하고, `캐릭터/MORI_상황별_캐릭터_설계.md`에서 그 UI에 맞는 기억 사서 실루엣과 복장, 전신·홍상 규격, 17개 런타임 상태, 기준 원본을 잠금 마스킹 편집 프롬프트를 작성했습니다. AI 생성 인상을 줄이기 위해 한 번에 여러 장을 생성하지 않고, 한 기준 홍상에서 표정·손·소품만 편집한 뒤 선회와 손을 수동 정리하도록 제작 절차를 제한했습니다.

실제 17장을 제작하는 과정에서 프롬프트 문구만으로는 안 잡히는 문제가 반복해서 나타났습니다. 배경을 "투명 알파 또는 균일한 단색"으로 열어두자 생성기가 하필 검정을 골랐는데, 캐릭터의 머리색·선회도 검정에 가까워 자동으로 배경을 지우면 머리카락에 구멍이 뚫리는 위험이 있었습니다. 이후 배경 요구사항을 "투명 알파만 허용, 예시 색상 없음"으로 좁히고, 정사각형 캔버스 요구를 참조 이미지에 종속되지 않도록 별도로 못박은 뒤에야 17장 모두 실제 투명 배경·1024×1024 정사각형으로 나왔습니다. 또한 감정 상태별 색 표식(예: 정답 카드의 초록 체크, 복구 카드의 초록 스티치)이 프롬프트에 있어도 빠지는 경우가 있어, 이후 프롬프트마다 "이 요소가 안 보이면 생성 실패로 간주하고 다시 그린다"는 조건을 추가해 재현율을 높였습니다. 완성된 이미지는 관찰(`observing`) 기준 홍상과 겹쳐 얼굴·머리핀 좌표가 흔들리지 않는지 확인한 뒤 실제 게임에 연결했습니다.

5. 외부 에셋과 오픈소스

항목	용도	버전	라이선스·출처
----	----	----	---------

Vite	개발 서버와 프로덕션 번들	8.2.0	MIT, https://github.com/vitejs/vite
OpenAI Codex	개발 보조	사용 환경 제공 버전	https://openai.com/codex/
JetBrains Mono	일지·터미널 텍스트용 모노스페이스 폰트, <code>public/fonts/</code> 에 자체 호스팅	v24 (가변 폰트)	SIL Open Font License 1.1, https://github.com/JetBrains/JetBrainsMono

MORI 전신 기준표와 17개 상황별 흉상 이미지는 참여자가 기록된 프롬프트를 사용해 ChatGPT Image에서 생성·선택했습니다. 폰트 1종(JetBrains Mono, 위 표 참고)을 제외하면 그 외 외부 이미지, 아이콘, 음원, 영상 파일은 사용하지 않았습니다. UI 그래픽은 HTML/CSS로, 효과음은 Web Audio API로 실시간 생성합니다.

6. 검증 명령

```
npm test
npm run build
STATELESS_GAME_URL=http://127.0.0.1:5173/ STATELESS_INPUT_MODE=keyboard npm run record
STATELESS_GAME_URL=http://127.0.0.1:5173/ STATELESS_INPUT_MODE=mouse npm run record
```

`npm test` 는 20개 게임 규칙·쿠키 직렬화 테스트를 실행합니다. `npm run build` 는 GitHub Pages에 배포할 `dist/` 정적 파일을 생성합니다.

`scripts/record-game.mjs` (`npm run record`)는 이전 SIGNAL LOCK 화면 구조(`#statement-board`, `.statement-chip` 등)를 자동으로 조작하도록 작성되어, 이번 SIGNAL TRIAGE 재설계로 해당 DOM 요소가 모두 제거되며 현재 실행할 수 없는 상태입니다. 재설계 검증은 별도의 임시 Puppeteer 스모크 테스트(마우스/키보드/ `prefers-reduced-motion` /전체 6웨이브 런)로 대체했으며, 녹화 스크립트를 새 화면 구조에 맞게 다시 쓰는 작업은 아직 진행하지 않았습니다.